

Training a PPO Agent to Play a Simple Platformer

Graduate Research Assistant Code Challenge | Code & Sorcery Lab, Ontario Tech University

1. Problem setup and MDP formulation

For this challenge I built a small Mario-style platformer in Pygame and then exposed the same game logic through a Gymnasium environment. The level has a clear start and goal, solid platforms, two pits, a moving enemy and a fixed hazard. The player moves under gravity, can run and jump, and dies by falling out of the level or colliding with an enemy or hazard. I kept the game deterministic so that a fixed seed and a fixed sequence of actions produce the same trajectory. The game can also run headless, which lets training execute without opening a Pygame window.

I chose a feature-vector observation instead of pixels. A pixel observation would have made the task more visually realistic, but it would also have introduced representation learning and a much larger sample requirement. For a first PPO experiment on a laptop, I wanted the learning problem to focus on control: can the agent learn when to move and jump using compact state information? The observation therefore has 11 values: player x and y position, horizontal direction, scaled vertical velocity, an on-ground flag, enemy x/y, hazard x/y and goal x/y.

MDP component	Definition	Reasoning
Observation	11-dimensional float vector.	Compact and cheap to train; includes the dynamic information needed for this fixed level.
Action	6 discrete actions: no-op, left, right, jump, left+jump, right+jump.	Keeps the action space small while allowing jumping and horizontal movement together.
Reward	0.01 x horizontal progress, +10 on goal, -10 on death.	Progress gives useful feedback before PPO discovers the sparse terminal reward.
Termination	Goal reached or player dies.	These are true endings of the task.
Truncation	1,000 environment steps.	Stops an agent that remains alive without finishing.

The progress reward was useful, but it also created a weakness that showed up in the experiment. An agent can earn some positive reward by moving forward even if it eventually gets stuck or fails to finish. That distinction between 'making progress' and 'solving the level' became important when I looked at the later checkpoints.

2. PPO method and training procedure

I used Stable-Baselines3 PPO with MlpPolicy. I did not supply a custom policy network, so this experiment uses the library's default PPO multilayer perceptron for vector observations: separate policy and value networks with two fully connected hidden layers of 64 units each. The policy network produces a distribution over the six discrete actions, while the value network estimates the expected return from the current state.

Before committing to a longer run, I used a short smoke test to check the full loop: environment reset, action selection, reward calculation, PPO updates, checkpoint evaluation and saved-model loading. The smoke run did not solve the level, but at roughly 20k training steps the agent reached about x=733. That

was already substantially farther than the random baseline and was enough evidence that the formulation was learnable.

Hyperparameter	Value
Random seed	42
Total training budget	204,800 timesteps
Evaluation interval	10,240 timesteps
Episodes per checkpoint evaluation	20
Learning rate	3e-4
n_steps	1,024
Batch size	64
Epochs per update	10
Gamma	0.99
GAE lambda	0.95
Clip range	0.2
Entropy coefficient	0.01

For the full run I evaluated the deterministic policy every 10,240 steps. The best checkpoint was chosen by completion rate, with mean reward used only as a tie-breaker. I made that choice before treating the final model as the result because task completion matters more here than collecting shaped progress reward. The final comparison used 100 episodes for the random policy and 100 episodes for the selected PPO policy.

3. Results

Policy	Mean reward	Success rate	Mean furthest x	Mean length
Random	-5.84	0%	131.8	697.1
Best PPO (30,720 steps)	18.63	100%	913	187

The random-action baseline never completed the level and, on average, only reached x=131.8. The selected PPO checkpoint was saved at 30,720 training steps. It reached x=913, completed the level and obtained a mean reward of 18.63. It also finished in 187 steps, compared with a mean episode length of 697.1 for the random policy.

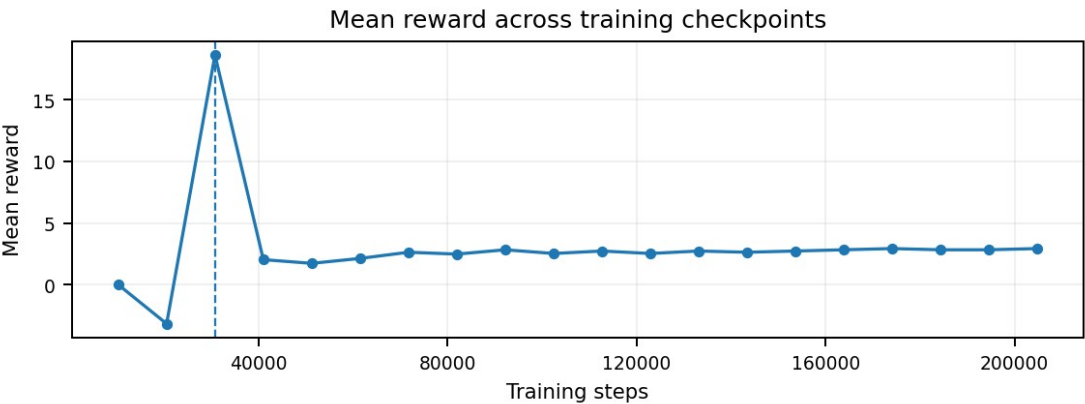


Figure 1. Mean reward at each evaluation checkpoint. The dashed line marks 30,720 steps, where the best policy was saved.

The reward curve shows a sharp improvement at 30,720 steps rather than a smooth climb throughout training. Mean reward rose to 18.63 at that checkpoint, then immediately dropped back close to the small positive values produced by partial forward progress. This is why looking only at the final training reward would have given a misleading picture of what the agent had learned.

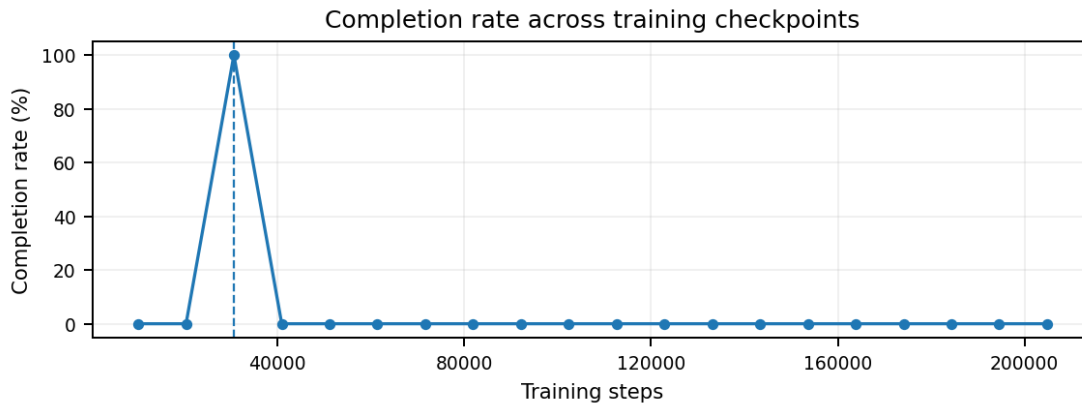


Figure 2. Completion rate across checkpoints. Only the 30,720-step checkpoint completed the fixed level during evaluation.

What did not work

The most interesting negative result was that training longer did not make the policy better. At 30,720 steps the policy completed the level in every deterministic evaluation episode. At the next checkpoint, 40,960 steps, completion fell back to 0%. The policy did not recover during the rest of the 204,800-step run.

I initially expected the learning curve to improve and then roughly stabilize. Instead, PPO found a successful behavior and later moved away from it. The shaped reward helps explain part of this: a policy can still receive positive return for moving some distance to the right, so losing the exact sequence required to clear the obstacles does not make every later policy look equally bad in reward terms. This result is also a reminder that PPO updates are not guaranteed to improve task-level performance monotonically.

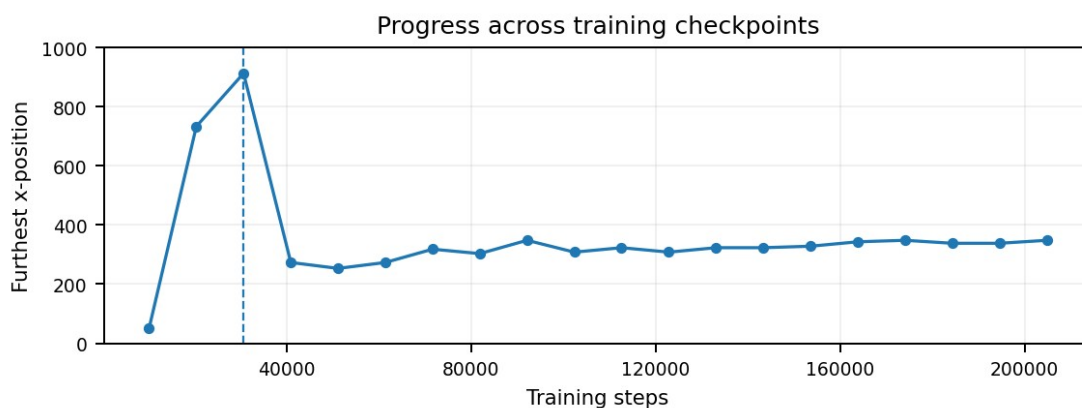


Figure 3. Furthest x-position reached. The early smoke run showed meaningful progress, the 30,720-step policy solved the level, and later checkpoints regressed.

The progress curve makes the failure easier to interpret. At 20,480 steps the agent reached $x=733$ but still failed at the later hazard. At 30,720 it reached $x=913$ and won. After that, furthest progress fell into roughly the $x=250$ -350 range. I kept this result rather than rerunning until I obtained a cleaner curve because the failure itself is useful evidence about the reward design and training stability. It also motivated saving and evaluating checkpoints instead of assuming that the final checkpoint was the best model.

4. Limitations and next steps

The biggest limitation is generalization. There is one fixed deterministic level, and the observation contains engineered state features rather than raw visual input. The static platform and gap geometry is not explicitly represented in the observation; because the level never changes, the player's position can indirectly act as a cue for where obstacles occur. For that reason I would describe the result as successful learning on this level, not as a general Mario-playing agent.

The evaluation setup is also deterministic. Running the selected policy for 100 episodes is useful as a reproducibility check, but those episodes repeat the same trajectory. The 100% completion result therefore means that the selected deterministic policy reliably solves this fixed deterministic level. It does not estimate robustness to different starting states, physics noise, enemy behavior or unseen layouts.

With another two weeks, my first experiment would be a small reward-shaping ablation. I would compare the current progress-shaped reward with a sparse goal/death reward and with a version that adds a small time penalty. I would measure both learning speed and whether a successful policy remains successful with continued training. I would also test a lower learning rate and different entropy coefficients because the post-30k collapse suggests that update stability is worth investigating.

After that, I would add variation to the environment: randomized starting positions, different enemy locations and several level layouts. I would train on a subset and evaluate on held-out layouts. With a larger research budget, I would run each configuration across multiple random seeds and report distributions or confidence intervals rather than relying on a single training trajectory. That would turn the current negative result into a more rigorous question: which reward and PPO settings produce policies that learn quickly and remain stable across seeds and level variations?

5. Reproducibility and disclosure

The repository includes a standalone training script, a separate evaluation/play script, a JSON configuration file, pinned dependencies, saved weights and machine-readable result files. The standalone training run reproduced the same best checkpoint at 30,720 steps and the same headline evaluation. The 204,800-step run took about 52.3 seconds on my MacBook Pro with an Apple M5 processor (10 cores) and 16 GB RAM using Python 3.13.15. I also created a fresh virtual environment from requirements.txt and successfully loaded and evaluated the saved model.

Libraries used were Pygame for the game and rendering, Gymnasium for the environment API, Stable-Baselines3 for PPO, PyTorch underneath Stable-Baselines3, NumPy for numerical data, Matplotlib for the plots and JupyterLab for exploratory experiments. I did not use an existing Mario RL environment or a pretrained agent.

Tutorials and learning resources. I referred to [Clear Code's Pygame tutorial](#) while learning Pygame/game-structure concepts, and [Johnny Code's custom Gymnasium environment / Stable-Baselines3 tutorial](#) while learning how a custom environment can be connected to reinforcement-learning tooling. I used these as learning references rather than copying an existing Mario environment.

I used ChatGPT as a development and learning assistant throughout the project. I used it to explain unfamiliar concepts and code, discuss the game and MDP design, debug implementation issues, structure the PPO experiments and reproducibility checks, and help edit the README and write-up. I ran and checked the project locally throughout: manual gameplay, Gymnasium environment validation, deterministic trajectory tests, PPO training, random-baseline comparison, saved-model loading, fresh-environment evaluation and rendered agent playback. I have kept the implementation small enough that I can explain the code and the design decisions directly.